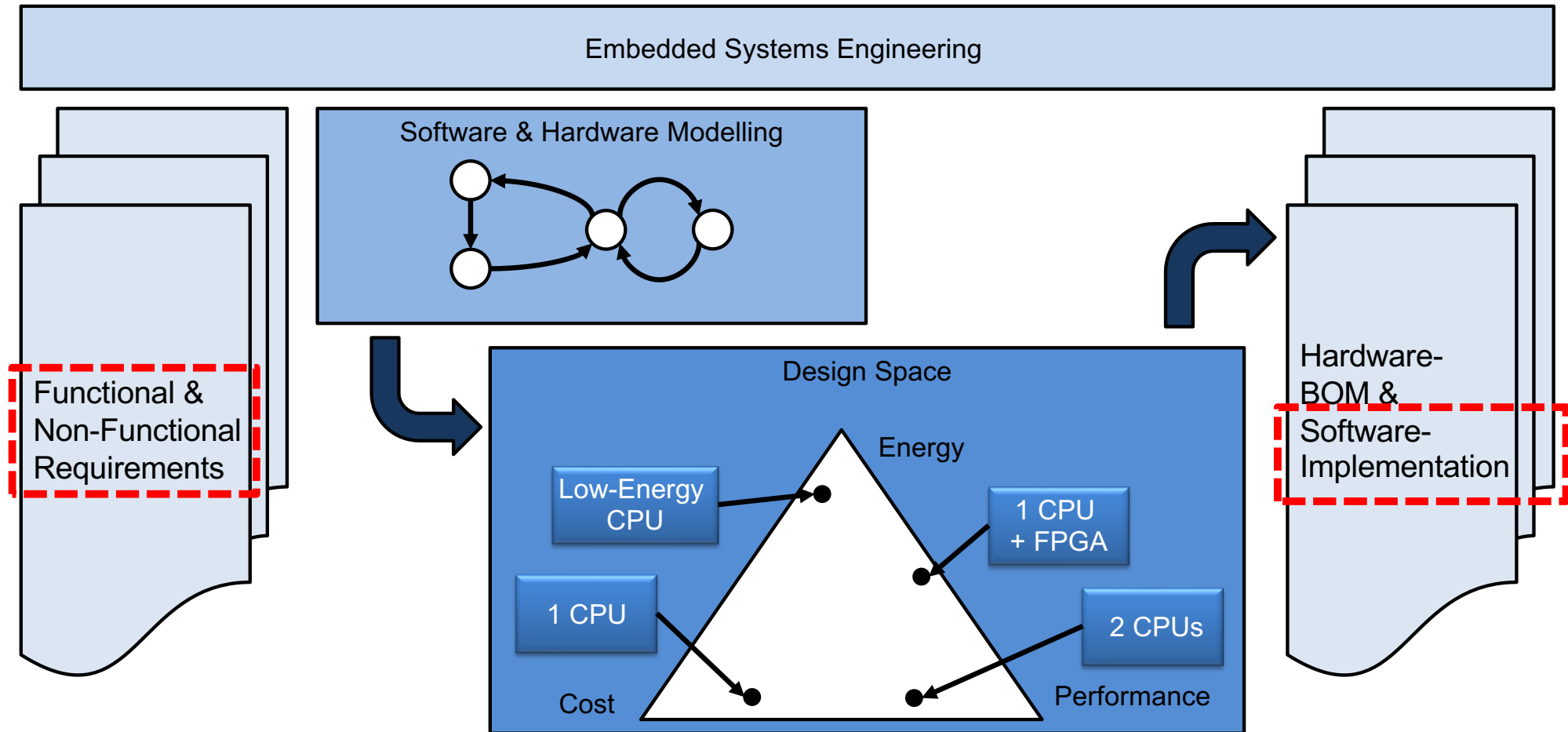


Embedded Software Engineering Real-Time Operating Systems

Simon Künzli, Tobias Welti
HS 2025

Advance Organizer ESE



Goals

After today's lesson, you should be able to ...

- **... elaborate on the duties and capabilities of an Embedded Operating System.**
- **... explain the differences in requirements for a general-purpose OS and an RTOS.**
- **... understand basic scheduling policies and apply them to examples to decide whether a feasible schedule can be found.**
- **... explain the difference between preemption and interrupts.**
- **... know the basic mechanisms for a task to preempt itself**

What does an OS do?



- <https://answergarden.ch/5098917>
- Manage hardware resources (...)
- Allow multitasking, pseudo-parallelism
- Handle access to peripherals, devices, network, using drivers
- Provide a user interface (graphical output, keyboard, mouse)
- Protect memory regions for every task



Why an Embedded OS?

- Embedded systems can require an OS if there are:
 - many parallel tasks to schedule,
 - many hardware resources to be shared
 - critical applications with high functionality requirements (medical applications, spaceships, robotics)
 - critical applications with very specific functionality (ABS in a car)
 - a broad range of functionality (smartphone)

Why not a General Purpose OS?

- A general-purpose OS (desktop) is not suitable:
 - too many unused features take up memory and processing time
 - not configurable, all functionality integrated
 - not made for critical applications
 - too much timing uncertainty: on a desktop computer, every task should get a roughly equal share of CPU time to provide a feeling of parallel execution. This does not suit embedded applications where some hard deadlines must be met.

Real-Time OS

Three essential requirements for RTOS

■ Predictable timing:

- The upper bound on the execution time of a task or a service must be known. (including ISR and service calls)
- guaranteed maximum latency

■ Meeting deadlines:

- The scheduler must be aware of deadlines.

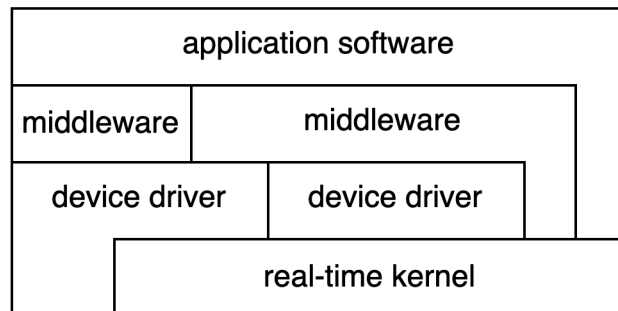
■ Managing priorities:

- Manage both hard and soft real-time requirements.
- Only a lean OS can guarantee hard deadlines. → configurability

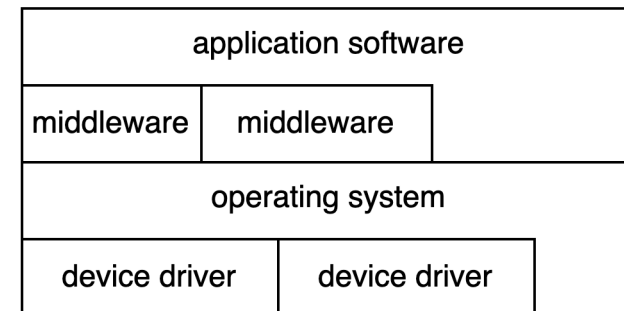
Embedded OS architecture

- **Access to underlying hardware through drivers.** Drivers on embedded OS are usually handled by the tasks. The kernel puts focus on scheduling.
 - More predictable timing: The scheduler handles the application, including access to hardware.
 - If multiple tasks need to access the same device, additional driver complexity is required, leading to more development costs.
 - Middleware can handle multiple tasks accessing the same device.

Embedded OS



General purpose OS



Real-Time Embedded OS features

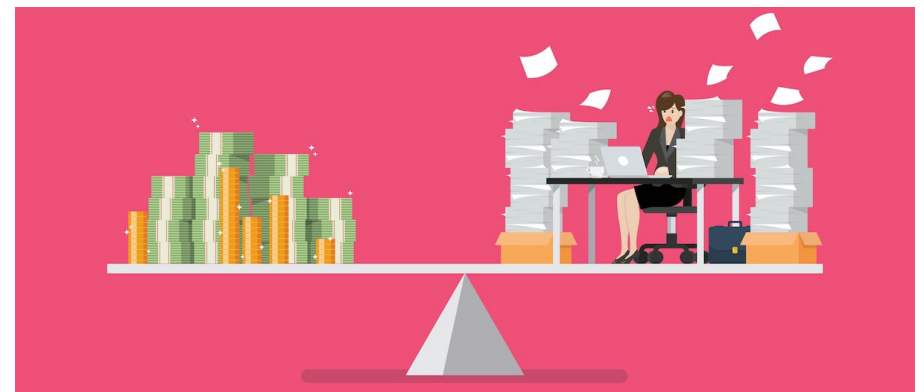
- In embedded systems, every task can perform an interrupt
 - A serious source of unreliability in a standard OS.
 - In embedded, the environment is usually well defined and filtered.
 - ➔ The software can (and should) be written accordingly.
- Tasks can be directly started through interrupts
 - The start address is stored in the interrupt vector table.
 - This allows shorter reaction times to start a task.
- Protection mechanisms may not be necessary
 - no untested programs should be present in ES
 - ➔ The software can be considered reliable
 - safety/security relevant systems need protection, but updates are difficult to distribute.



Why not do it yourself?

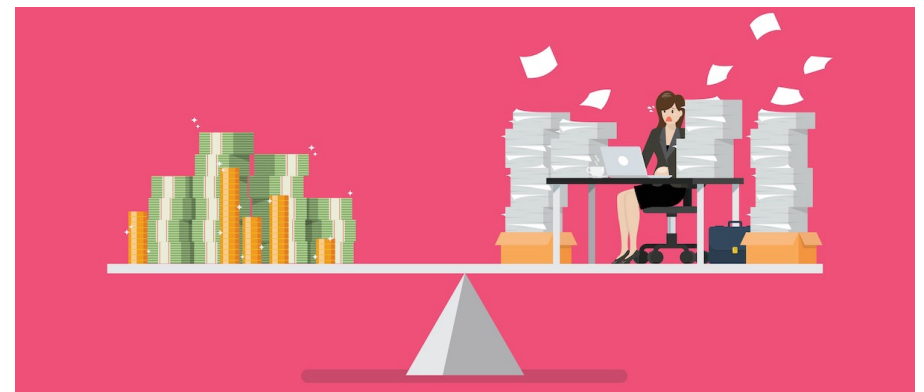
Who has programmed...

- their own scheduler?
- a filesystem?
- a device driver?



Why not do it yourself?

- RTOS are tested, configurable, and performant. The main work is done
- Task scheduling is not hard, it could be written by yourselves, but...
 - it needs to be tested
 - will it be as performant and lean?
 - is it worth the effort?
 - there's more to it than scheduling...
- cost of usage vs cost of development
- Real-time OS are configurable and tested for different architectures and platforms
 - remove unneeded functionality
 - switch platforms more easily



Main functionality of the kernel

Task management

- **Quasi-parallelism:** multiple procedures running in parallel as processes or threads
 - handling process states, context switches,
 - allowing preemptive task switching and interrupt handling
- **Scheduling:** deadlines must be guaranteed.
 - Fair distribution of resources is often secondary.
- **Inter-Process Communication**
- **Support for real-time clock: system ticks**
 - regular intervals generated by a hardware timer
 - generate a beat for the scheduler
- **Task synchronization:** some tasks may depend on each other
 - allow critical sections,
 - wait for another task to finish some work before you can go on
 - semaphores in real-time OS are deeply integrated into scheduling to prevent deadlocks

Task States

■ Running:

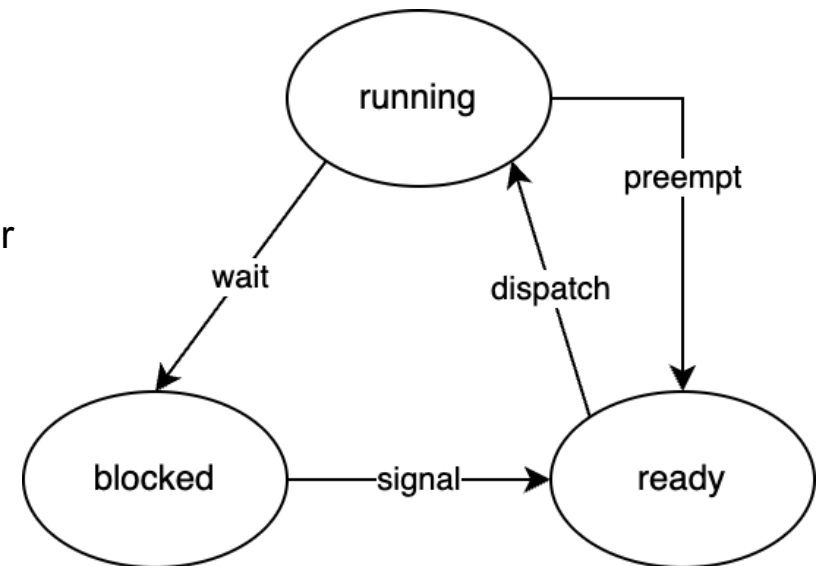
- only one task can be running.
- task blocks when it needs to wait for resources
→ task can call wait() to pass CPU back to scheduler
- scheduler can preempt a task

■ Ready:

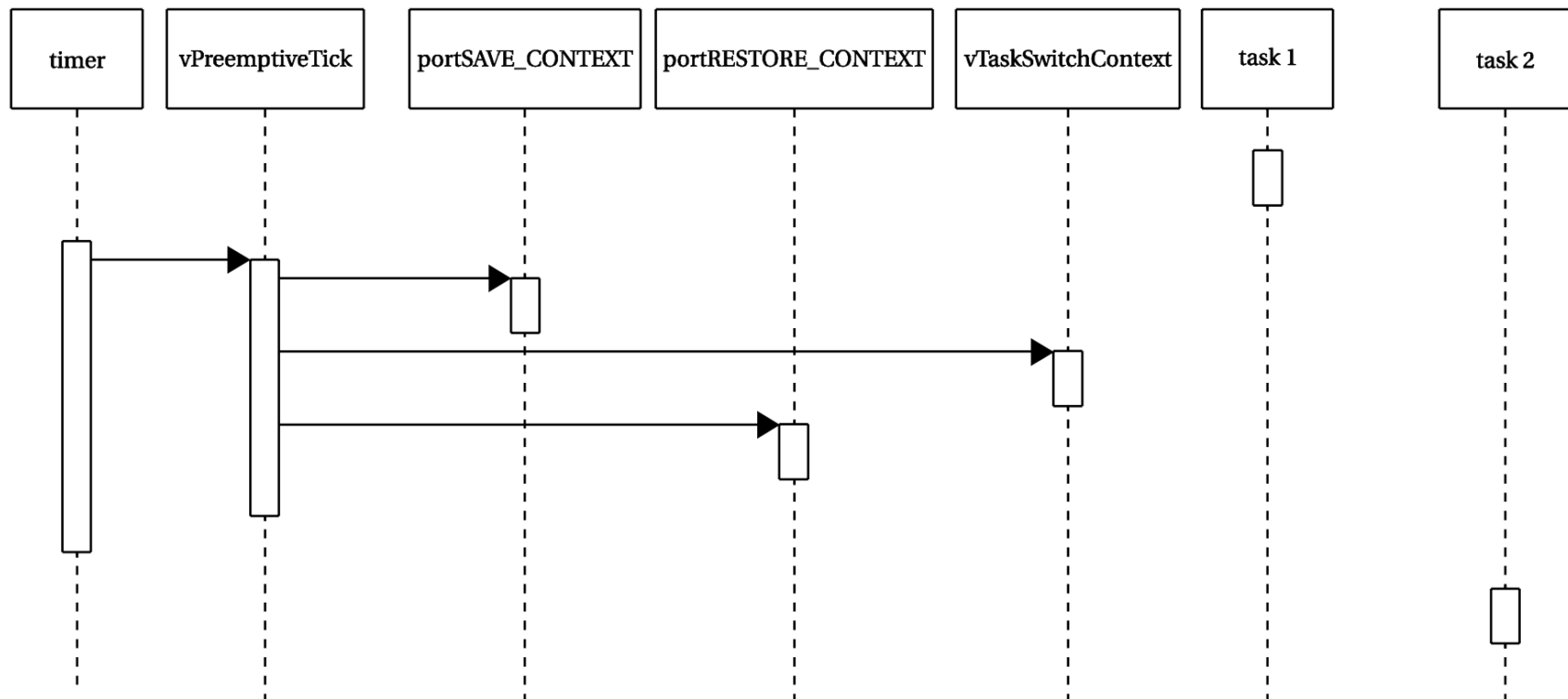
- queue for tasks waiting to run
- scheduler selects the next task to run (dispatch)

■ Blocked:

- task is waiting for a resource or an event
- system tick can cause task to change to ready (when waiting)
- task can be unblocked when resource becomes available



Preemption



Preemption

- **System tick:**
 - triggers preemption of current task

- **Context switch:** takes time
 - storing context
 - selecting next task
 - restoring context

- **Tasks unaware of preemption**
 - expect context as it was when preempted

Preemption vs Interrupts

- **Preemption:** caused by system tick. Tick is a regular interrupt from a hardware timer. The system tick is an interrupt service routine that takes care of switching to the next ready thread.
 - Tick caused by hardware timer
 - ISR preempts running thread, selects next task from ready queue
 - Executes context switch and sets task to running state
- **Interrupt:** can be caused by external sources, at any time
 - Asynchronous event
 - Causes context switch for ISR
 - Execution usually returns to the same task after handling of ISR.
- **Common:**
 - running task has no way of knowing when a context switch will occur.
➔ Problems?

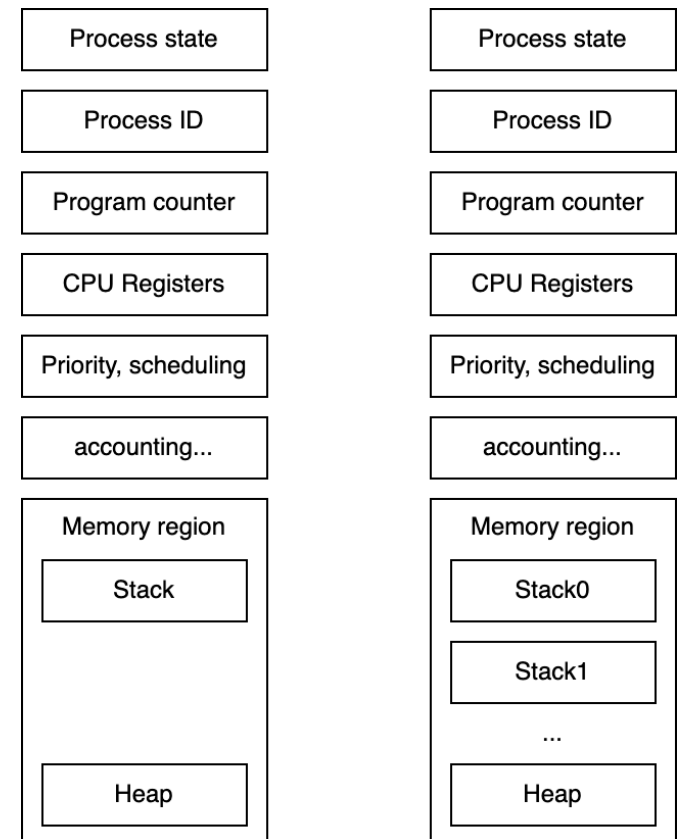
Keeping order

- OS needs to keep track of all tasks on the system
- This includes the following information:

Task context: process control block

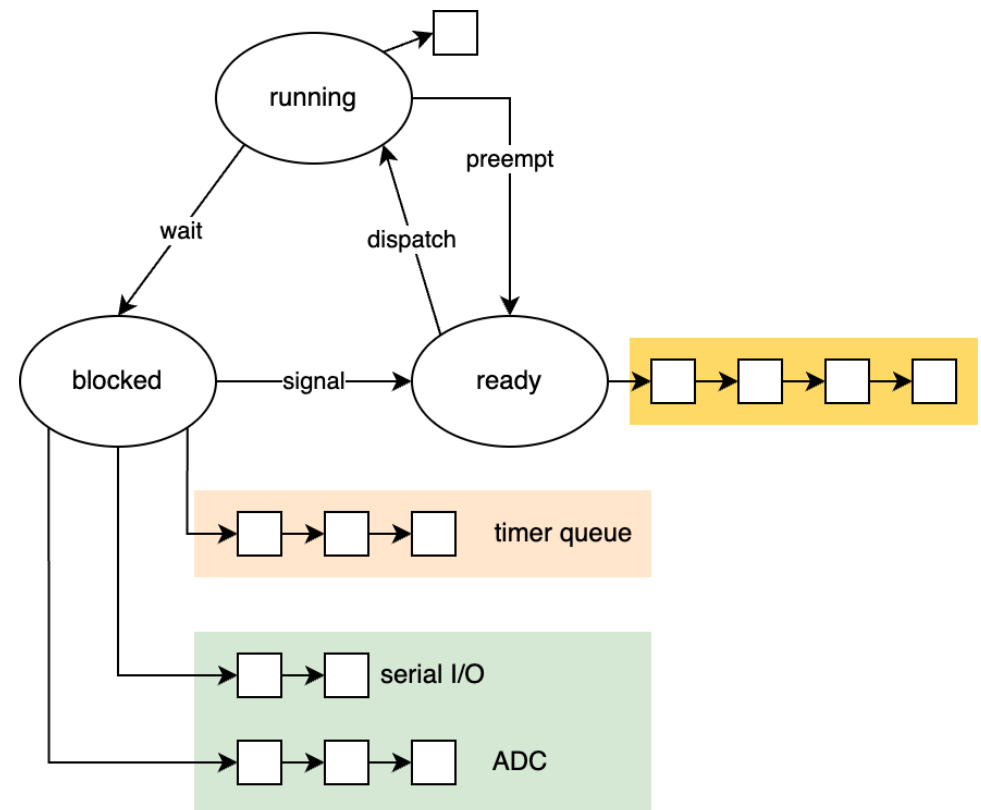
Book-keeping on processes:

- Process control block (PCB) stores process context
 - memory regions, location of stack, heap
 - CPU registers, PC, scheduling info
- Process can have one or multiple threads
 - multiple threads sharing PCB some resources (memory, peripheral access)
 - individual storage for thread's stack, registers.
- Threads advantages:
 - faster context switch between threads.
 - no major intervention of OS for thread switch needed.



Threads

- OS keeps track of threads using PCBs
- PCBs for every state kept in queues assigned to state



Use cases for RTOS

- Resource handling through Interprocess Communication (IPC)
- Scheduled execution at specified times
- Task priorities
- Scheduling in general
- Scheduling metrics

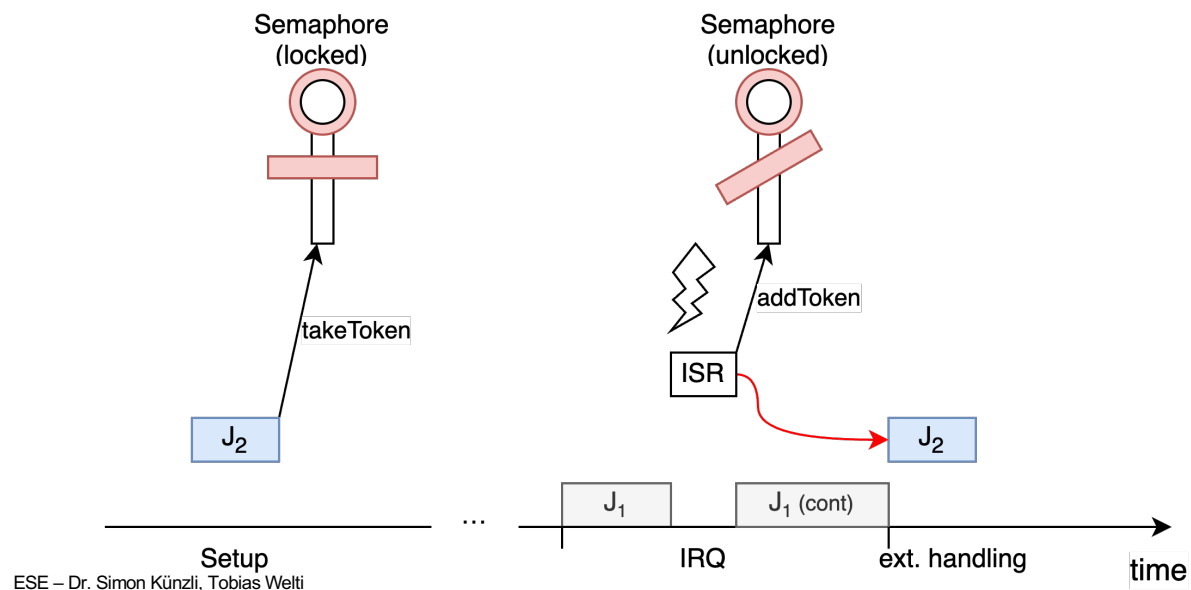
Interprocess Communication

- Semaphore:
 - a signal to prevent entry into a critical section if a resource required in this section is already busy
- can be used to set up a thread as an extended interrupt handler:
 - ISR handles only time-critical section
 - A separate task will handle time-uncritical section



Extended Interrupt Handling

- **Setup:** J2 is started and sends itself to blocking state by trying to take the token of the semaphore.
- Normal operation until IRQ (J1 running)
- ISR resets IRQ and adds token, unblocking J2 → becomes ready
- System tick preempts J1 and J2 will run

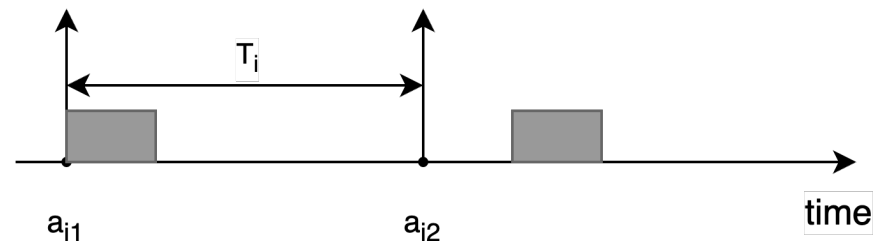


Timed execution

- System tick: hardware timer sending IRQ, i.e. every 10 us
- Thread loops can be executed regularly using OS timer interfaces

```
while (true) {  
    // loop body  
  
    ...  
  
    // ask to be woken again in 200 ms  
    delay(ms_to_ticks(200)); // convert 200 ms to system ticks  
}
```

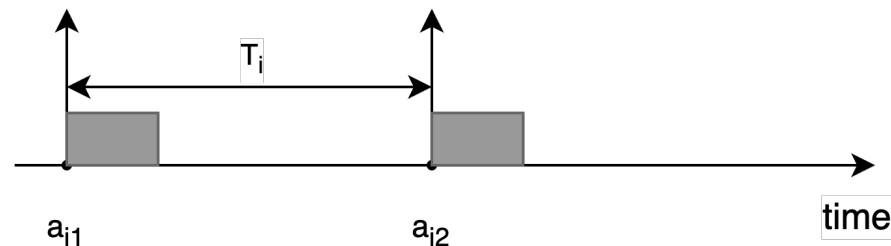
- Problem?



Timed execution

- Problem: adding full period time at the end of the loop body will result in longer period
- OS can help, keeping track of last process waking time

```
while (true) {  
    // loop body  
    ...  
    // add 200 ms to last wake-up time  
    delayUntil(&lastWakeTime, ms_to_ticks(200));  
}
```



Priorities

- **Example:** two threads, equal priorities. Threads do not call *wait()* themselves

```
while (true) {  
    ... // do something  
    for(i = 0; i < 100000; i++){...}  
    // no call to delay() or wait()!  
}
```

- **OS scheduler will preempt thread at System Tick.**
Thread is enqueued in **ready**, next thread can run.



Priorities

- **Example:** two threads, different priorities. Threads **do** preempt themselves

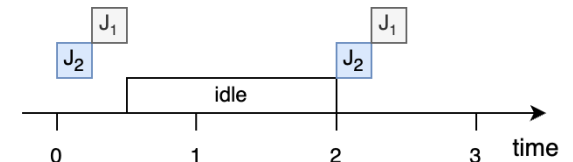
J2 started with prio 2 (high)

J1 started with prio 1 (low)

```
while (true) {  
    ... // do something  
    delayUntil(&lastWakeTime, us_to_ticks(2));  
}
```

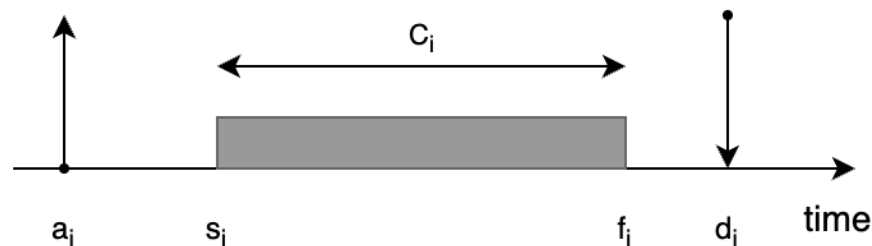
- **OS scheduler will go to idle when ready queue is empty. → energy-saving state until next System Tick**

When threads J1 and J2 are enqueued in **ready**, next thread can run, scheduler chooses higher priority first.



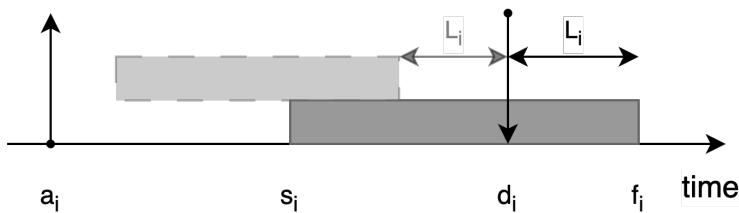
Scheduling: Definitions

- **Arrival/release time** a_i : when a task becomes ready for execution
- **Computation time** C_i : how much time a task needs on the processor until finished
- **Deadline** d_i : by what time a task must be finished
- **Start time** s_i : time when task starts to be executed
- **Finishing time** f_i : time when a task finishes its execution
- **Feasibility**: whether a task can be completed within their constraints
- **Schedulable**: if a set of tasks has a feasible schedule



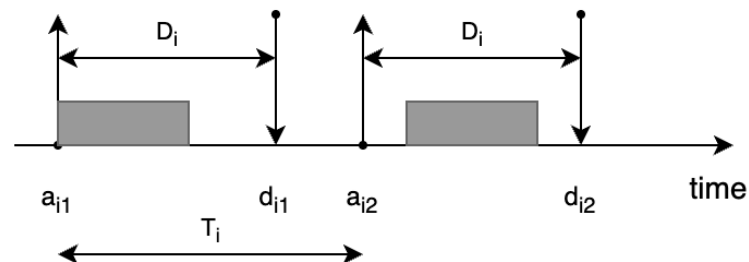
Scheduling: Definitions

- **Lateness:** $L_i = f_i - d_i$ Can be negative if the task finished before the deadline
- **Exceeding time:** $E_i = \max(0, L_i)$ also “Tardiness”: Time a task is active after its deadline
- **Slack time:** $X_i = d_i - a_i - C_i$ How long a task can be delayed before starting so it still meets the deadline

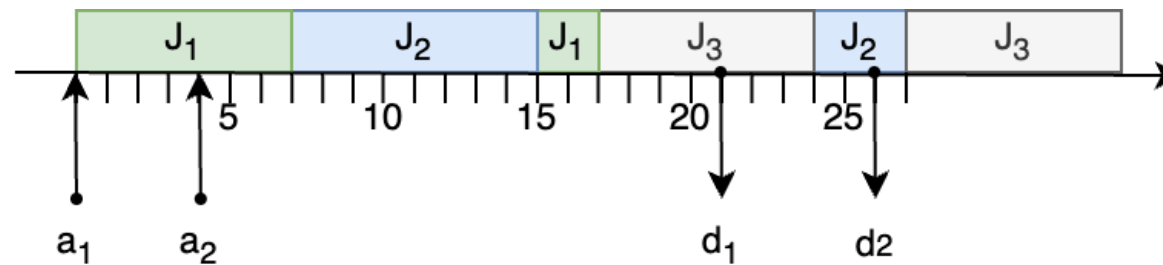


Scheduling

- **Periodic tasks:** a task to be repeated at regular intervals, or periods T_i
- Phase Φ_i is the activation time of the first instance
- **CPU utilization:** $\frac{C_i}{T_i}$ Usually given in percent

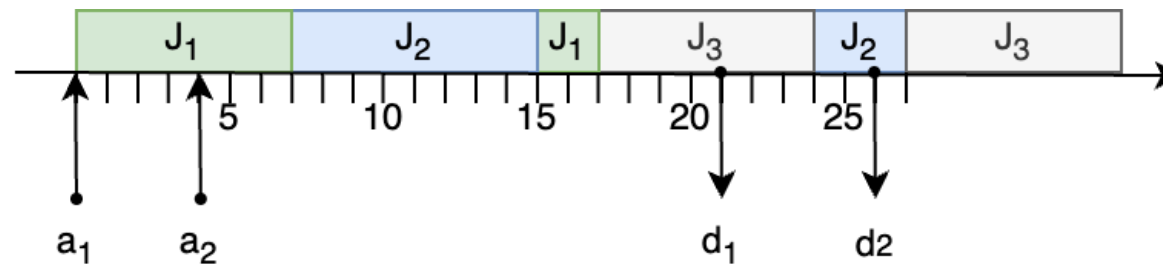


Example



	J ₁	J ₂
Start time, s_i		
Finishing time, f_i		
Computation time, C_i		
Lateness, L_i		
Tardiness, E_i		
Slack, X_i		

Example



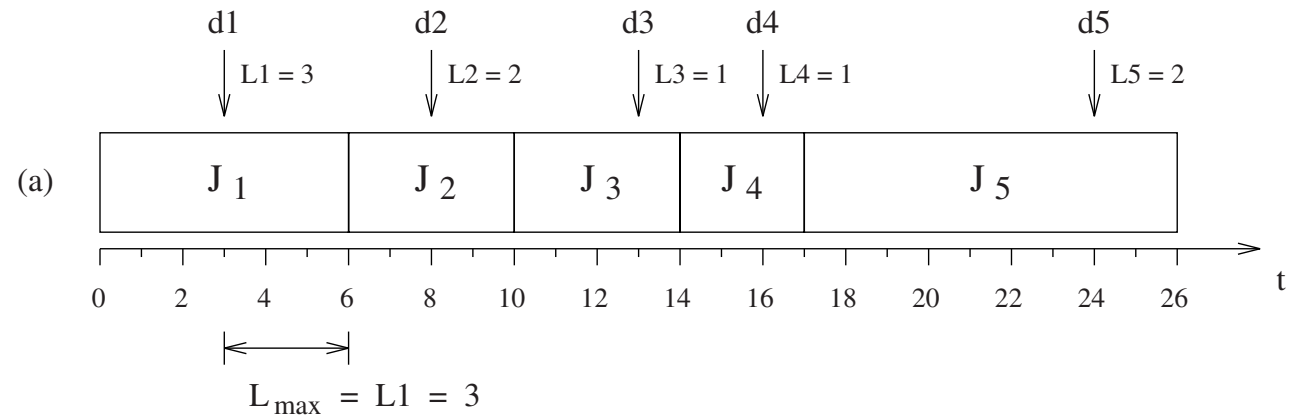
	J₁	J₂
Start time, s_i	0	7
Finishing time, f_i	17	27
Computation time, C_i	9	11
Lateness, L_i	- 4	1
Tardiness, E_i	0	1
Slack, X_i	12	11

Scheduling Metrics

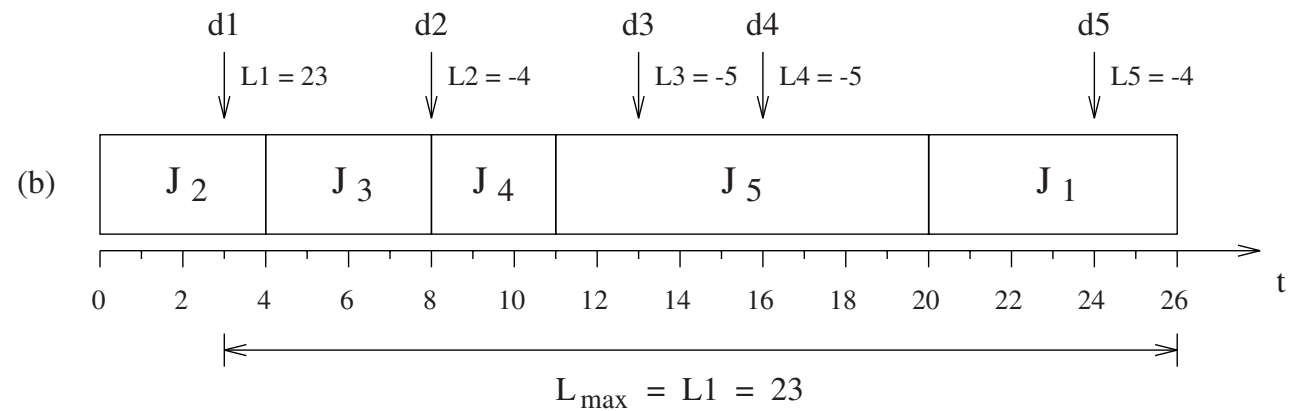
- Average response time: $t_r = \frac{1}{n} \sum_{k=1}^n (f_{ik} - a_{ik})$
- Total completion time: $t_c = \max_i(f_i) - \min_i(a_i)$
- Maximum lateness: $L_{max} = \max_i(f_i - d_i)$
- Number of late tasks: $N_{late} = \sum_{i=1}^n miss(f_i)$
- Discuss: Which metrics are suitable for real-time systems? Why?

Scheduling Metrics Example

maximum lateness is minimized

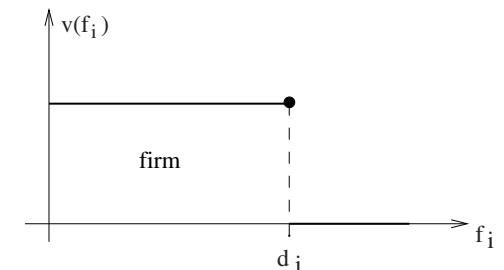
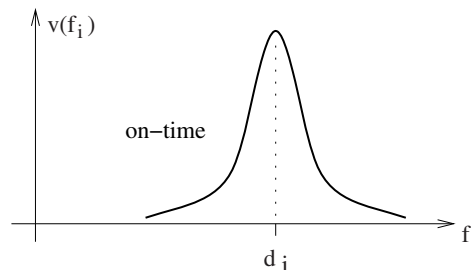
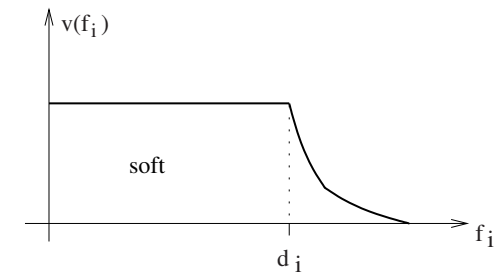
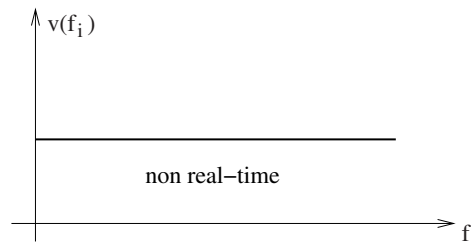


number of task misses minimized



Scheduling cost functions

- **non real-time:** has no deadline, therefore no dependence on finishing time
- **soft:** noncritical deadline, the value of the system only decreases slowly when deadline is missed. Early termination is no problem.
- **on-time:** finishing as close to the deadline as possible is desired. Non-zero lateness decreases value. Example: Audio recorder
- **hard (firm):** system is worthless if it misses a deadline.



Examples of scheduling algorithms

■ Rate monotonic scheduling (RMS)

- simple rules
- fixed priorities, according to their periods
- fully analyzable
- lean, but wasteful

■ Earliest Deadline First (EDF)

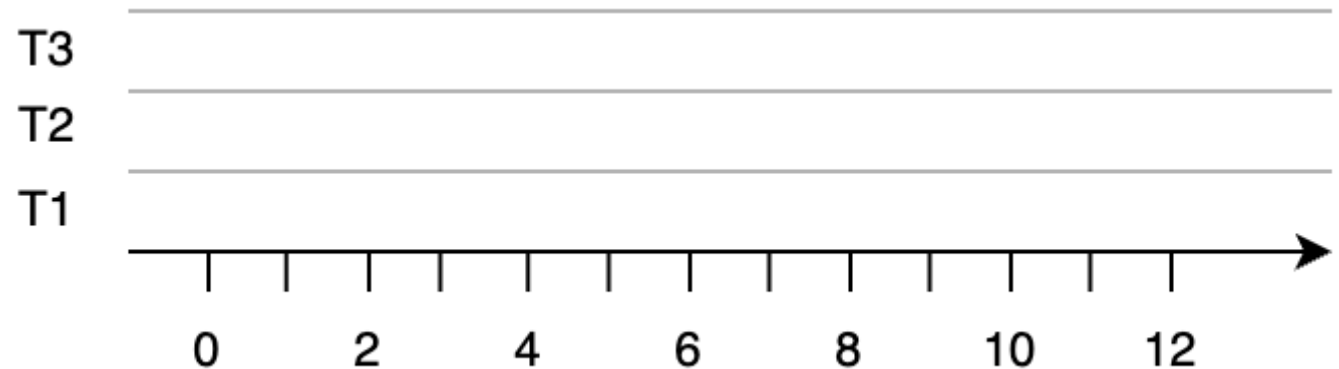
- dynamic priorities, according to their deadline
- works for periodic as well as aperiodic tasks
- schedulable if sum of CPU utilization is less than 100%
- can have significant overhead to select next thread.

Rate monotonic scheduling (RMS)

- RMS (Liu and Layland) is a widely-used, analyzable scheduling policy
 - all threads run on a single CPU
 - context switch time is considered zero
 - Threads have no dependencies between each other
 - Computation time is constant for each thread
 - Deadline is at end of its period
 - Processes with shorter periods have higher priority
 - Highest-priority ready process runs
- RMS gives an optimal priority assignment (fixed priorities)
- No better fixed-priority scheme than RMS
 - if RMS does not give a feasible schedule, there is no feasible schedule using fixed priorities!
- **But:** RMS is only guaranteed to work up to total CPU utilization of less than 69 %
 - for higher utilization, draw out the schedule until it repeats.
 - ➔ up to 30 % unused CPU

Example: RMS

	T1	T2	T3
Computation time	1	2	1
Deadline = Period	3	4	6

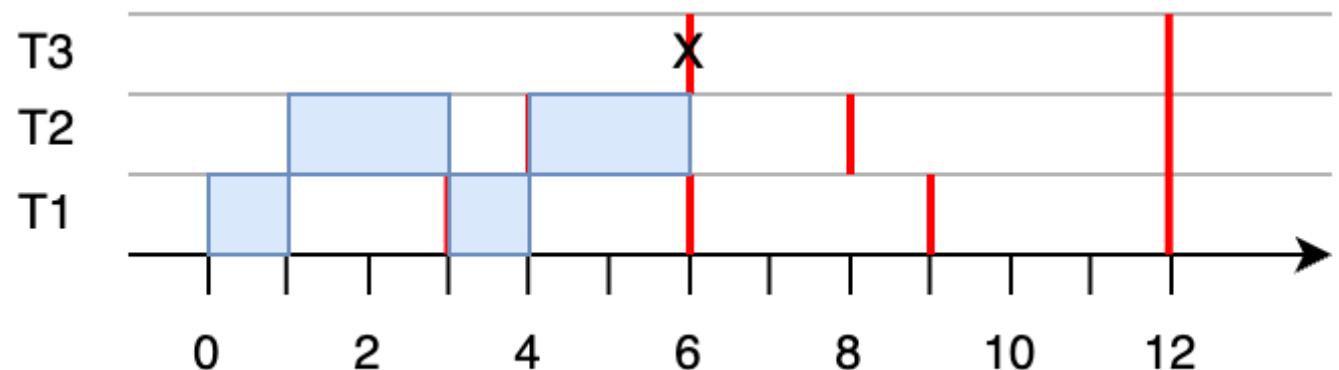


1. Critical moment (start here): all tasks are ready
2. Assign priorities: shorted period has highest prio.
3. Draw deadlines for all threads
4. Repeat: Start highest prio as soon as a task is done.

CPU Utilization = ?
Schedule repeats after ____ ticks

Example: RMS solution

	T1	T2	T3
Computation time	1	2	1
Deadline = Period	3	4	6



1. Critical moment (start here): all tasks are ready
2. Assign priorities: shorted period has highest prio.
3. Draw deadlines for all threads
4. Repeat: Start highest prio

$$\text{CPU Utilization} = 4/12 + 6/12 + 2/12 = 12/12 = 100 \%$$

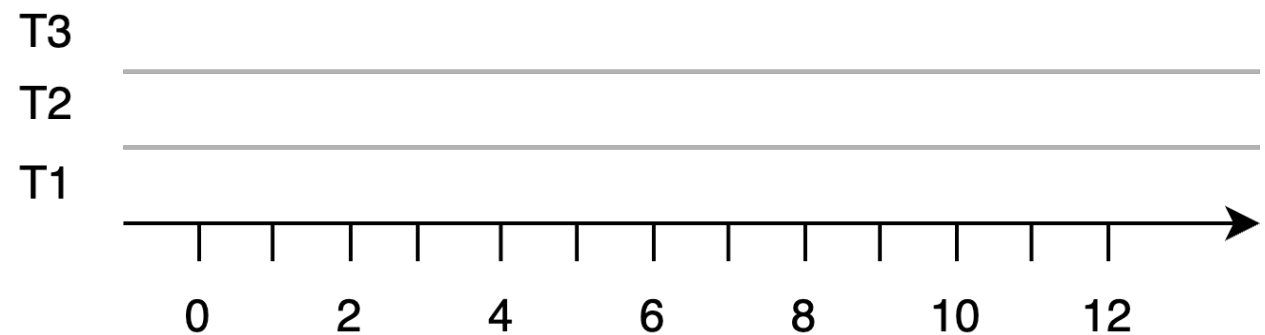
Deadline pattern repeats after 12 ticks. **Not schedulable with RMS**

Earliest Deadline First (EDF)

- **Dynamic Priority scheduling scheme**
 - Thread closest to its deadline gets highest priority
 - Need to recalculate thread priorities at every timer interrupt
- **EDF guarantees feasible schedule for set of threads with CPU utilization $\leq 100\%$**

Example: EDF

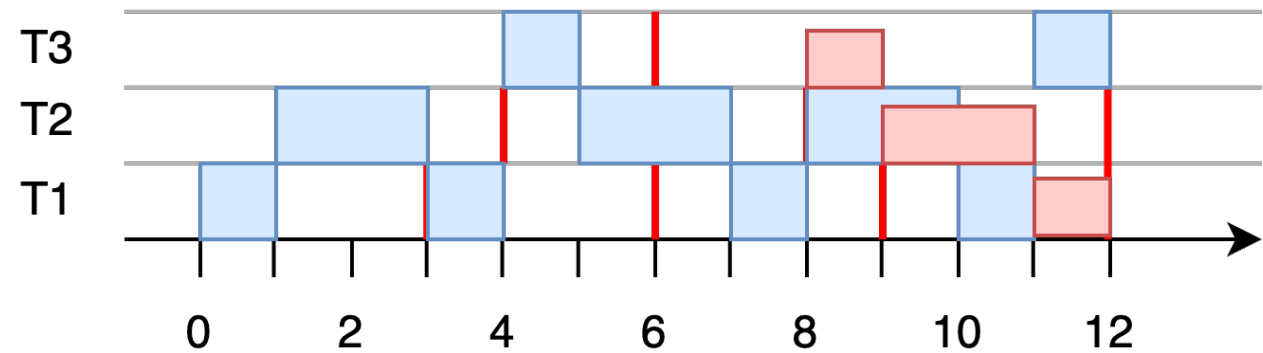
	T1	T2	T3
Computation time	1	2	1
Deadline = Period	3	4	6



1. Draw all deadlines
2. Calculate priorities: Closest deadline has highest prio.
3. Run thread with highest prio.
4. Goto 2.

Example: EDF solution

	T1	T2	T3
Computation time	1	2	1
Deadline = Period	3	4	6



Thread closest to its deadline gets highest priority

EDF Implementation

- On each timer interrupt:
 - Compute time to deadline for each thread
 - Run thread closest to deadline
- Generally, too expensive for practical use
- EDF guarantees meeting all thread deadlines for CPU utilization $\leq 100\%$
 - Only if time for scheduling and context switches is zero!
 - If system tick is fine-grained (short System Ticks), scheduling overhead can use more CPU than thread set.

Scheduling concerns

■ Thread context switching time:

- EDF and RMS assume zero time for context switching
- Developer must make sure that context switching time is significantly smaller than thread periods.
→ can be tricky in a real system
- non-zero context switch time can disturb a tight schedule
- effects are difficult to predict, order of switches can affect switching time
- Usually, context switch uses a few hundred clock cycles. Keep tick duration significantly higher.

■ What if set of threads is not schedulable?

- Change requirements
- Reduce execution times
- Get a faster CPU

Scheduling anomalies (preemptive)

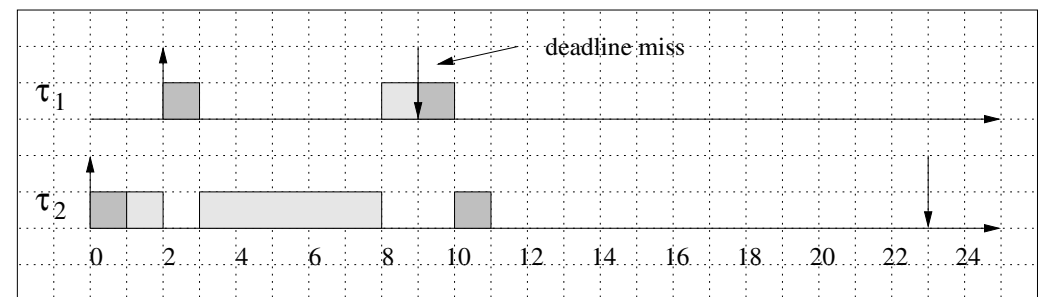
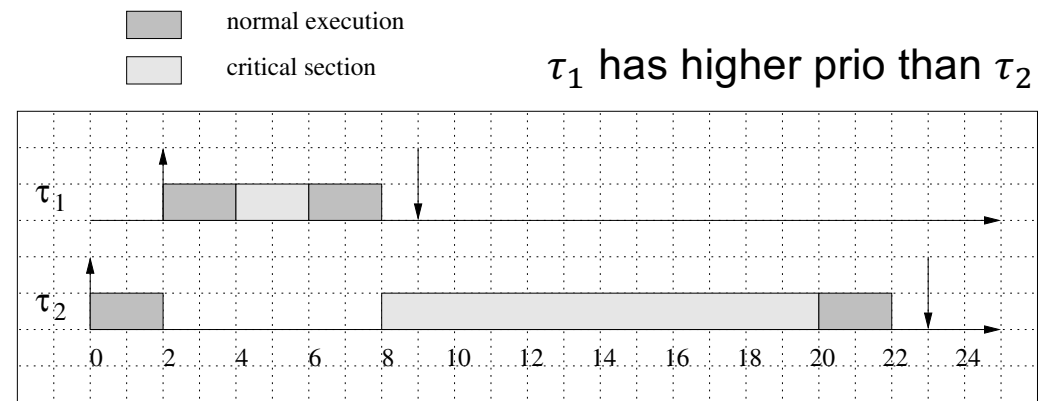
Increasing processor speed can cause missed deadlines:

τ_1 and τ_2 share a resource in their critical section.

a) τ_1 can preempt τ_2 , because τ_1 arrives during normal execution of τ_2 .

b) Because processor runs faster, τ_2 has already reached the critical section, blocking the resource. τ_1 will run up to its critical section, then wait for the resource.

→ τ_1 will miss the deadline.



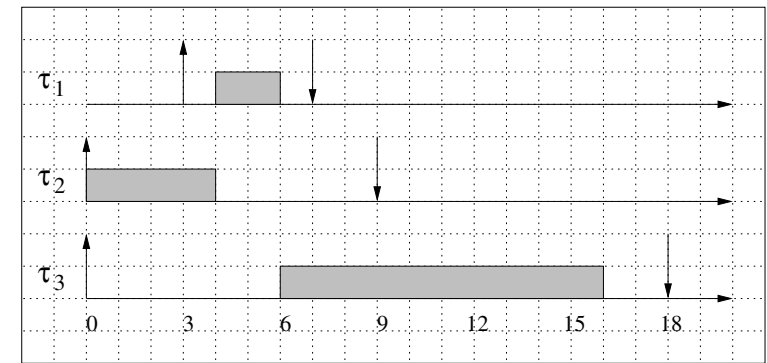
Scheduling anomalies (non-preemptive)

τ_2 and τ_3 arrive at $t=0$,

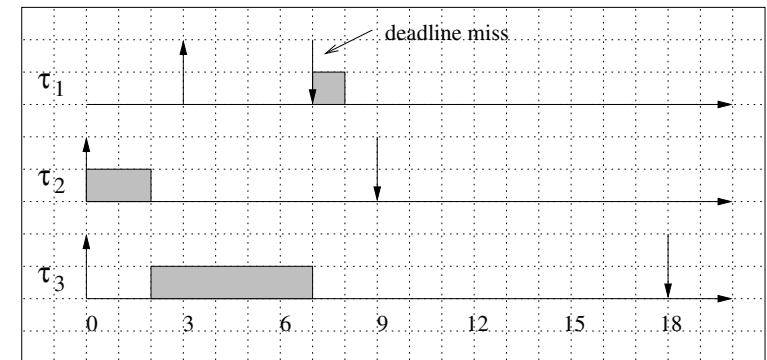
τ_1 arrives at $t=4$

Priority scheme: closest deadline next.

- a) τ_1 arrives during execution of τ_2 and is enqueued in the ready queue. Because it has higher priority than τ_3 , τ_1 is selected for running and meets its deadline
- b) On a faster processor, τ_3 starts running before the arrival time of τ_1 , which in consequence misses the deadline.



(a)



(b)

Summary

- **Real-time OS must guarantee deadlines and provide reliable timing while being fast.**
- **Safety and security are often secondary in ES: if there is only tested software on the device, and no access to computing resources from outside.**
- **RTOS provides multi-tasking and guaranteed scheduling.**
- **Choice of scheduling metric depends on application.**
- **Scheduling and deadlines can be hard to test. Anomalies can occur.**
- **Scheduling principles can be simple but wasteful**